

Advanced Network Programming VIII

Hybrid protocols

Juliusz Chroboczek

26 November 2025

1 Client-server and peer-to-peer

Client-server is a familiar, well-understood structure, and has a number of advantages. First of all, it is efficient: recovering data most often requires a single request, unlike other structures that might require searching across a network. It is easy to implement, and, most importantly, it is reasonably easy to debug: the server has a global vision of all the activities in the application, and is therefore the right point to implement logging (“debugging printf’s”). Implementability and debuggability are important features of a protocol.

There are, however, good reasons to want to use peer-to-peer communication. Server infrastructure is costly: the server needs to be paid for, the clients have already been bought. Client-server suffers from linear scaling, while well-done peer-to-peer is logarithmic. In a client-server system the server is a single point of failure, which makes it vulnerable to failures and attacks, while a well-implemented peer-to-peer system can survive the loss of a significant fraction of nodes.

2 Hybrid protocols

Some things, however, are difficult to do in a pure peer-to-peer system. The main difficulty is in *rendez-vous*¹: the issue of locating the peer that is responsible for a given resource (for example a given nickname in a chat protocol, or a given file in a file transfer protocol). Other difficult issues are the distribution of cryptographic keys, and NAT traversal, which requires synchronisation between peers that are unable to communicate.

It is possible to solve these problems in a pure peer-to-peer protocol. For example, *rendez-vous* can be achieved by using a *Distributed Hash Table* (DHT) algorithm such as Kademlia, key distribution can be secured by using a *Certificate Authority* (CA). However, it is much simpler and more efficient to use a server (or multiple servers) for this kind of *control* activities.

A protocol that uses both peer-to-peer (for user data) and a client-server architecture is called a *hybrid* protocol (Figure ??). The client-server protocol is generally used for control of the peer-to-peer protocol, and is therefore called the *control* protocol.

1. Pronounced *rendey voo*.

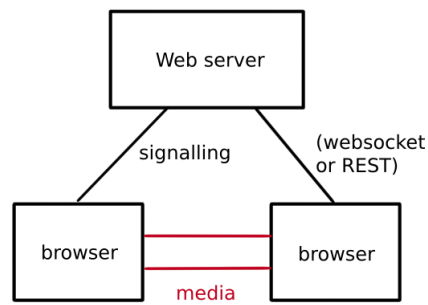


Figure 1. A hybrid videoconferencing system

2.1 The control protocol

In a hybrid protocol, the client-server protocol is called the *control protocol*.

The control protocol will usually be used to solve rendez-vous. The usual solution is to have the clients authenticate themselves with the server and then publish the socket addresses (IP + port number) at which they are reachable. The server will then distribute the contact information to the other clients.

Another possible function is the distribution of cryptographic keys. Just like IP addresses, clients' public keys are published on the server, and communicated to other peers. Public keys are public information, no encryption is necessary, but they need to be bound to a given client, and must therefore be subject to authentication and integrity protocols. (Another solution is to have the server act as a certifying authority, which avoids the need to contact the server in order to obtain a peer's public key.)

The control server is trusted From a security point of view, the control server must be trusted: since it controls the distribution of both IP addresses and cryptographic material, it can trivially mount a man-in-the-middle attack.

The control server needs to scale The control server carries a load that is at least $O(n)$ in the number of connected clients. However, the constants are minuscule, so scaling is not usually an issue.

The control server is a single point of failure In most implementations, the peer-to-peer network cannot function without the control protocol, which is therefore a single point of failure. Hybrid protocols therefore have some of the same issues as centralised systems: it is vulnerable to overloading, vulnerable to denial of service, and vulnerable to legal attacks. However, as the server is much simpler and carries much less load than the server of a centralised system, it can more easily be made robust.

3 Examples

3.1 BitTorrent file sharing

File sharing in the BitTorrent protocol is bootstrapped by transferring a *torrent file* out-of-band (typically through a web server). The torrent file contains a degenerate Merkle tree which is used by receivers to check the integrity of the data being transferred.

The root hash of the Merkle tree is called the torrent's *Info-Hash*, and is used for authentication, both in the client-server protocol and in the peer-to-peer protocol. In effect, BitTorrent allows anonymous participation, but requires *a priori* knowledge of the Info-Hash.

BitTorrent, as originally defined, consists of two subprotocols:

- the *tracker protocol*, which is used for communicating with a control server (known as a *tracker*);
- the peer-to-peer file transfer protocol.

3.1.1 The tracker protocol

BitTorrent's control protocol is minimalistic. The client sends a request to the tracker over HTTP that includes the Info-Hash and the port number that the client uses for the peer-to-peer protocol. The server derives a socket address by combining the source address of the request with the port number, and stores it in a database. It then returns a list of 50 peer addresses to the client.

The server forgets about a client 30 minutes after it last contacted it. A client that needs to remain available will typically recontact the server every 28 minutes.

3.1.2 The peer-to-peer protocol

BitTorrent peers periodically attempt connections to other peers. When a peer connection succeeds, the peers exchange a handshake during which the connecting peer proves knowledge of the Info-Hash. The data itself has its integrity protected by the Merkle tree contained in the torrent file.

3.1.3 Evolution of BitTorrent

BitTorrent is an extensible protocol, and it has evolved over the years. The most notable evolution happened in reaction to attacks (legal and otherwise) against trackers: modern implementations of BitTorrent perform peer discovery using a combination of two peer-to-peer protocols, a distributed hashtable (DHT) protocol called *Kademlia*, and a simple gossip protocol called *Peer Exchange* (PEX). With these changes, BitTorrent has become a pure peer-to-peer protocol.

BitTorrent traffic is now optionally encrypted², with a session key that is dynamically negotiated using plain Diffie-Hellman authenticated by the Info-Hash. Finally, BitTorrent is now carried by μ TP, a dedicated transport protocol implemented over UDP, which uses a custom congestion controller (TCP congestion control is too aggressive for BitTorrent).

2. Please enable encryption in your BitTorrent client.

3.2 Peer-to-peer videoconferencing

There are two kinds of videoconferencing protocols: centralised protocols, such as *Zoom*, *Webex* or *Galene*, where users' media (audio and video) are sent to the server and redistributed to other clients, and peer-to-peer protocols, such as *Google Duo*, where the media are exchanged directly between clients. Peer-to-peer protocols don't scale well (every client needs to send $n - 1$ flows), but they provide the lowest latency and therefore the best quality.

Peer-to-peer videoconferencing uses a control server that has a wide range of services:

- authentication and authorisation;
- rendez-vous;
- exchange of cryptographic keys, both for mutual authentication of peers and end-to-end encryption;
- negotiation of transport parameters (transport-layer protocols, port numbers, fallback to tunnelling etc.);
- negotiation of media parameters (codecs, etc.);
- NAT traversal (STUN).

3.3 Comparison

	BitTorrent	videoconferencing server
client authentication	info-hash in URL	password, token, etc.
session information	n/a	SDP through server
p2p authentication	info-hash in handshake	key exchange through server
end-to-end encryption	ad-hoc	key exchange through server
end-to-end integrity	out-of-band info-hash	key exchange through server
transport parms	through server	through server
NAT traversal	peer-to-peer	through server (STUN)
tunnelling fallback	n/a	negotiated through server

Figure 2. Comparison of hybrid protocols

The two approaches outlined above are at opposite ends of the spectrum (Figure 2). The BitTorrent tracker provides minimal service (just rendez-vous), and has been replaced by peer-to-peer protocols. The videoconferencing control server is maximalistic, it essentially implements the full protocol with the exception of media transfer.

My personal explanation is the BitTorrent is optimised for low cost and ease of server deployment, while videoconferencing is optimised for speed of connection and reliability. It is also possible that the different designs are simply a historical accident.